RMTC 两个人怎么分工（通俗版）

> 目的：不重复造轮子，10 天把程序做出来
>
> 版本：简单版，2026-10-21

---

0. 一句话

不要一个人做第一阶段、一个人做第二阶段。

因为两个阶段都要做同样的事：收裁判消息、看视野、记地图、找路、发动作。 如果按阶段分，两个人一定会各写一套，最后合不起来。

正确做法：

> **一个人做“底座”，一个人做“策略”。先一起把底座搭好，再各做各的。**

---

1. 先把程序想成两层

第一层：底座

就是程序的地基和水电：

- 跟裁判通信：收“视野报告”，发“动作”；
- 看懂视野：知道自己在哪、周围有什么；
- 记地图：哪里走过、哪里有障碍、哪里有资源；
- 找路：从 A 点走到 B 点，躲开障碍；
- 发动作：把“往前走”变成裁判能看懂的消息。

底座只有一套，两个人都用。

第二层：策略

就是“车脑子里的想法”：

- 第一阶段：先去哪、捡哪个资源；
- 第二阶段：先做哪个任务、要不要打架、要不要跑；
- 大模型（LLM，就是聊天机器人）说“那边危险”，车要不要躲；

- 血量低的时候，是打还是跑。

策略可以分两块，两个人各管一块。

---

2. 两个人怎么分

下面用 A、B 代称，按你们谁更擅长什么来换名字。

谁	主要负责	说人话
A	底座 + 第一阶段	管“车怎么跟裁判说话、怎么看路、怎么找资源”
B	第二阶段 + 大模型	管“怎么做任务、怎么打架、怎么听大模型建议”
两个人一起	定规矩、合并代码、测试、报告、现场	管“别各写各的，最后能拼起来”

A 具体做什么

- 收裁判的“视野报告”，把里面的信息看懂；
- 把 9×9 小地图里的“相对位置”换成大地图上的“绝对坐标”；
- 记住障碍、资源、走过的地方；
- 写找最短路线的方法（BFS，就是“找路”）；
- 写第一阶段“看到资源就过去捡”的策略；
- 做测试工具、打包程序、做镜像（Docker）。

B 具体做什么

- 第二阶段的任务：2 个普通资源 + 1 个高价值资源 + 到达指定坐标；
- 战斗：什么时候打、什么时候防、什么时候跑；
- 大模型：怎么问、怎么解析回答、回答不对怎么办；
- 随机事件：爆炸、世界分裂、辣椒这些，让大模型帮忙判断危险；
- 第二阶段调优：怎么在 500 Tick 里拿更多分。

两个人一起做什么

- 第一天一起把“最小骨架”跑通；
- 一起定数据格式和文件分工；
- 每天把代码合到一起；

- 互相看代码，10 分钟就够；
- 一起跑完整测试；
- 一起写技术报告；
- 现场一个操作、一个观察。

---

3. 第一天一起做什么（最重要）

不要第一天就分头写。两个人花 2–3 小时一起做这几件事：

1. **建一个代码文件夹**（一个仓库），不要各建一个。
2. **一起定“数据长什么样”**：
   - 裁判发来的消息里有哪些字段；
   - 自己位置、血量、存活、资源、障碍、敌人、任务、事件文本；
   - 动作只有四种：走、打、防、等。
3. **一起跑通最小版本**：
   - 收到裁判消息；
   - 发一个“等待”；
   - 下一帧能看到“上一动作已执行”的回执。
4. **约定谁改哪个文件**，写在一张纸上或文档里。

这一步看起来慢，其实最省时间。 因为后面两个人就不用猜“你那边到底怎么写的”。

---

4. 代码大概怎么放

不用很复杂，能分开就行：

```
rmtc_bot/
  main.py          # 只负责：收消息 -> 调用策略 -> 发动作
  models.py        # 两个人共用的数据格式
  ros_io.py        # 跟裁判通信
  perception.py    # 看懂视野
  memory.py        # 记地图
  planner.py       # 找路
  policies/
    stage1.py      # A 写：第一阶段
    stage2.py      # B 写：第二阶段
  combat.py        # B 写：打架
  llm/             # B 写：大模型
```

分工原则：

- A 主要改：`ros_io.py`、`perception.py`、`memory.py`、`planner.py`、`stage1.py`；
- B 主要改：`stage2.py`、`combat.py`、`llm/`；
- `main.py`、`models.py`、`config.py` 两个人共用，但**一次只让一个人改**；
- **不要复制粘贴**。B 要用 A 的找路功能，就直接调用，不要自己再写一份。

---

5. 怎么同时开发，不互相等

方法一：用“假数据”

B 不用等 A 把裁判通信写完。 B 可以自己写几条“假的视野报告”，比如：

- 自己在 [1,1]；
- 旁边有个资源；
- 远处有个敌人；
- 任务要求去 [12,12]。

然后拿这些假数据调第二阶段的策略。 等 A 的底座好了，再换成真数据。

方法二：用“假大模型”

A 不用等 B 把真大模型接好。 先让程序调用一个“假模型”，它永远返回固定的一句话，比如：

> “先去 [12,12]，路上别打架。”

这样 A 可以先测试“程序会不会被大模型卡住”。 等 B 接好真模型，再换掉。

方法三：分开测试，不要抢同一个车位

两个人同时开练习端时，建议：

- A 用 `ROS_DOMAIN_ID=27`，网页端口 `8080`；
- B 用 `ROS_DOMAIN_ID=37`，网页端口 `8081`；
- 不要两个人同时控制同一个席位（比如都用 A 席位）。

不然车会乱动，日志也会混在一起。

方法四：每天合并，不要攒到最后

- 每天至少把代码合到一起一次；
- 保证合完还能跑；
- 不要“我写一周，最后一天再合”；

- 小步合并，出了问题容易找。

方法五：一个人管一个文件

- 不要两个人同时改同一个文件；
- 要改别人的文件，先说一声；
- 改公共文件（`main.py`、`models.py`、`config.py`）之前，先发消息：“我要改这个，可以吗？”

方法六：改“接口”要先说

比如 A 想给视野报告加一个字段，B 的策略可能就会坏。 所以规则是：

> **先告诉对方，再改代码；改完一起跑一遍。**

---

6. 10 天简单安排

日期	A 做什么	B 做什么
10/21	一起搭骨架，跑通“收消息、发等待、收回执”	一起定数据格式，写假数据
10/22–24	看懂视野、记地图、找路、第一阶段找资源	第二阶段任务、资源、战斗（先用假数据）
10/25–27	第一阶段调优、测试工具	第二阶段联调、死亡复活、事件避让
10/28–29	打包、镜像、换席位测试	接真大模型、调提示词、做失败备用方案
10/30	一起完整测试、冻结代码	一起完整测试、写报告
10/31	录视频、备份、提交	报告校对、提交
11/1	现场操作	现场看分数、看日志、和裁判沟通

时间不够就一起保：

> **通信 → 第一阶段 → 第二阶段任务和资源 → 大模型调用 → 打架 → 复杂事件**

---

7. 报告和现场怎么分

技术报告：一份，不要两份

- A 写：程序整体结构、通信、看视野、记地图、找路、第一阶段、测试记录；
- B 写：第二阶段任务、战斗、大模型、随机事件、第二阶段结果；
- 一起写：分工、时间安排、遇到的问题、怎么解决、总结；

- 最后指定一个人统一格式，不要两种文风。

现场：一个操作，一个观察

- **A 操作**：设席位、设网络、设令牌、启动程序、按规则重启；
- **B 观察**：看分数、看任务、看血量、看大模型成功次数、看裁判消息；
- 队长负责申请“策略切换”；
- 两个人都不能手动控制车，只能让程序自己跑；
- 上场前写一张小纸条：启动命令、环境变量、检查项。两个人都要会。

---

8. 千万不要这样做

1. **各写一个程序**：一个 stage1 bot，一个 stage2 bot。

   → 一定会重复写收消息、找路、发动作。

2. **各写一套找路**：B 觉得 A 的不好用，自己复制一份改。

   → 最后两套逻辑不一样，很难查问题。

3. **各调一套大模型提示词**。

   → 只留一份，B 主改，A 帮忙看。

4. **两个人同时改同一个文件**。

   → 很容易互相覆盖。

5. **用同一个 ROS 域、同一个端口、同一个席位测试**。

   → 车会乱，日志会混。

6. **平时不合并，最后一天合**。

   → 最后一定一堆冲突。

7. **一个人等另一个人“写完再开始”**。

   → 用假数据、假模型，先并行。

8. **报告各写各的**。

   → 一份文档，分节写，一个人统稿。

---

9. 一张纸检查清单

- ☐ 一个代码文件夹，一个程序
- ☐ 第一天一起跑通“收消息 → 发等待 → 收回执”
- ☐ 数据格式写下来，两个人都同意
- ☐ A 管底座和第一阶段
- ☐ B 管第二阶段和大模型
- ☐ 用假数据、假模型并行开发
- ☐ 每天合并一次
- ☐ 每次改公共文件前先说一声
- ☐ 每周至少一次完整练习
- ☐ 10/30 冻结代码
- ☐ 10/31 20:00 前交材料

---

10. 最后再说一遍

> **底座一套，策略两块；先一起搭骨架，再分头写；每天合，不复制。**

两个人不是“各做一半”，而是“一个人把地基打好，另一个人把楼上装修好，但楼梯和电线是共用的”。